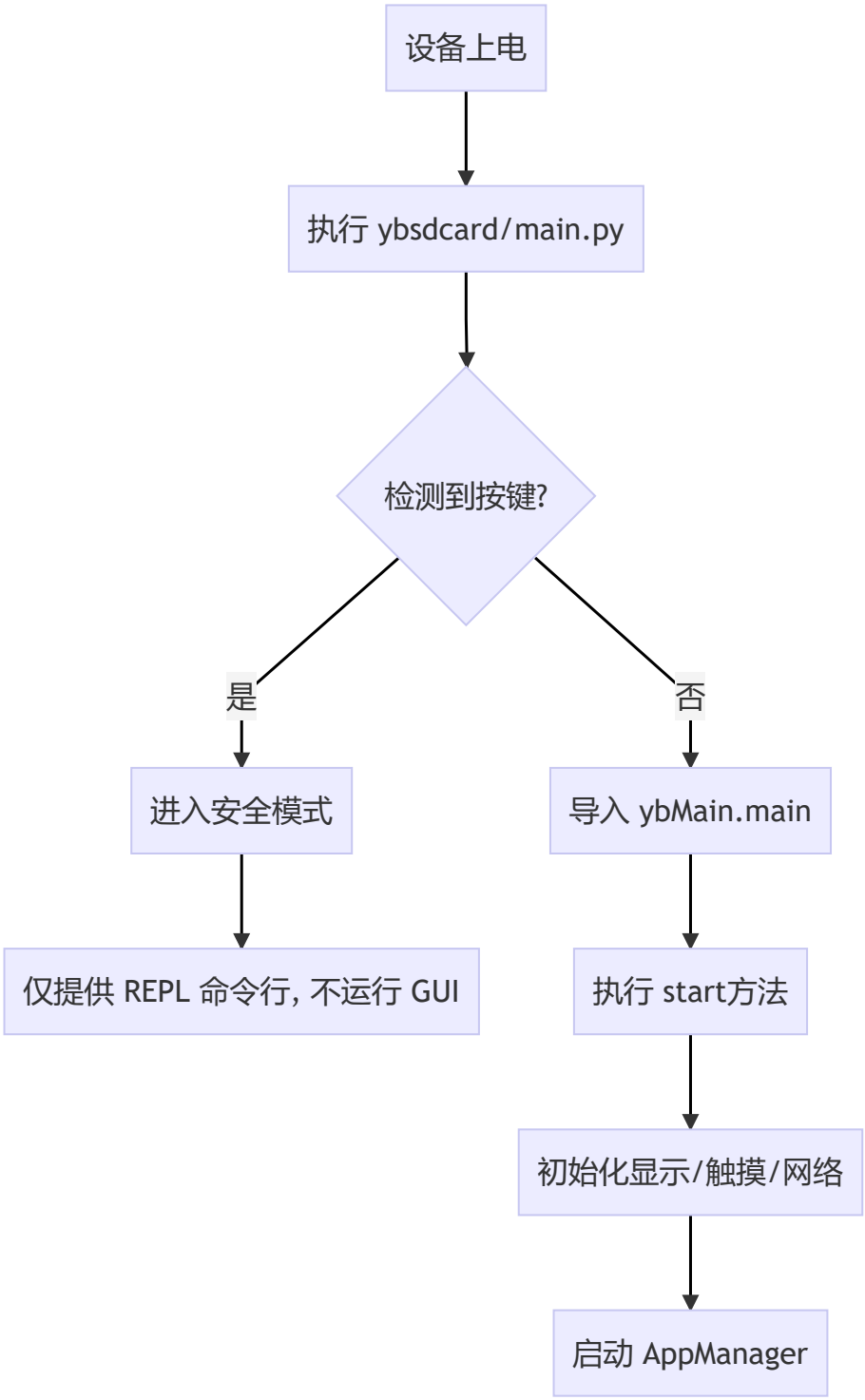
GUI 系统架构

GUI 系统架构
- 1. 课程目标
- 2. 架构图解
- 3. 核心源码分析
  - 3.1 安全启动机制 (Anti-Brick Logic)
  - 3.2 启动核心引擎

1. 课程目标

本节课我们将分析 Yahboom K230 GUI 系统的整体架构

2. 架构图解



3. 核心源码分析

文件路径: `src/ybsdcard/main.py`

这个文件是整个 GUI 系统的**入口点**。当设备开机时，MicroPython 固件会自动寻找并运行 `main.py`。

3.1 安全启动机制 (Anti-Brick Logic)

如果你希望临时的不要开机自启GUI程序，又或者是 `main.py` 里出现了严重的 Bug，导致设备一开机就崩溃或卡死，无法连接 IDE 进行修复，你可以通过在上电前长按三秒Key键来跳过自启（上电前按下 --> 上电 --> 三秒后松开）。

K230 的这套 GUI 系统设计了一个安全后门**：

```
import time
from ybUtils.YbKey import YbKey

# 初始化按键
key = YbKey()
abort_main = 0

# 检测按键状态
if key.is_pressed():
    time.sleep_ms(10) # 消抖
    if key.is_pressed():
        abort_main = 1 # 如果按键被按下，标记为中止启动
```

代码解读:

1. `YbKey()`: 系统初始化了一个按键对象。
2. `key.is_pressed()`: 在启动的瞬间，检测用户是否按下了功能键。
3. `abort_main = 1`: 如果检测到按下，变量置为 1，意味着“不要启动 GUI，把控制权还给用户”。

3.2 启动核心引擎

如果用户没有按下功能键，系统将继续加载 GUI 核心：

```
if not abort_main:
    from ybMain.main import *
    start()
```

代码解读:

1. `if not abort_main`: 只有在安全检查通过后才执行。
2. `from ybMain.main import *`: 导入核心模块 `ybMain`。这就像是 Linux 内核加载完毕，开始启动 `systemd` 或图形界面。
3. `start()`: 这是核心引擎的启动函数，一旦执行，屏幕将点亮，进入图形界面。